
DOCUMENTO DE POLÍTICAS DE SEGURIDAD

Sistema de Gestión de Cirugías — *HospiTEC*

Curso: CE-1115 Seguridad de la Información
Semestre: I Semestre 2026
Profesor: MSc. Andrés Vargas Rivera
Institución: Instituto Tecnológico de Costa Rica
Versión: 1.0
Fecha: Abril 2026
Integrantes: Ricardo Borbón Mena
Jorge Guillén Campos
Felipe Jiménez Ulate
Carlos Rodríguez Segura
José María Vindas Ortiz

Índice

1. Alcance	3
1.1. Sistemas y componentes cubiertos	3
1.2. Roles de usuario en el sistema	3
2. Principios de Seguridad	4
2.1. Confidencialidad	4
2.2. Integridad	4
2.3. Disponibilidad	5
3. Principios de Implementación de Seguridad	5
3.1. Mínimo privilegio	5
3.2. Seguridad por capas (<i>Defense in Depth</i>)	5
3.3. Fallos seguros (<i>Fail Secure</i>)	6
4. Manejo de Datos Sensibles	6
4.1. Clasificación de datos	6
4.2. Datos de salud	6
4.3. Documentos PDF	7
5. Política de Contraseñas	7
5.1. Almacenamiento	7
5.2. Política mínima de contraseñas	7
6. Política de Autenticación y Sesiones	7
6.1. Flujo de autenticación	7
6.2. Gestión de sesiones (JWT + base de datos)	8
6.3. Almacenamiento del token en el cliente	8
6.4. Protección contra fuerza bruta	9
7. Control de Acceso	9
7.1. Principio de mínimo privilegio	9
7.2. Validación en el backend	9
8. Protección contra Vulnerabilidades Web	10
8.1. Inyección SQL	10
8.2. Cross-Site Scripting (XSS)	10
8.3. Cross-Site Request Forgery (CSRF)	10
8.4. Cabeceras de seguridad HTTP	10
9. Registro y Auditoría	10
9.1. Sistema de auditoría	10
9.2. Eventos auditados	11
9.3. Protección de datos en los logs	11

10. Configuración Segura del Entorno	11
10.1. Variables de entorno	11
10.2. Docker Compose	12
10.3. Manejo de errores	12
11. Cumplimiento con OWASP Top 10	12
12. Referencias	13

1 Alcance

Este documento establece las políticas de seguridad aplicables al **Sistema de Gestión de Cirugías del Hospital TEC**, una aplicación web desarrollada para el proyecto semestral del curso CE-1115. El sistema permite la gestión integral de citas quirúrgicas, incluyendo el agendamiento, asignación de personal médico, manejo de expedientes de pacientes y carga de documentos clínicos en formato PDF.

1.1 Sistemas y componentes cubiertos

- **Frontend:** Aplicación de página única (SPA) desarrollada en React (Node.js), que expone las interfaces de usuario según el rol asignado.
- **Backend:** API REST desarrollada con FastAPI (Python), responsable de toda la lógica de negocio, autenticación, autorización y acceso a datos.
- **Base de datos:** PostgreSQL, que almacena usuarios, cirugías, sesiones, registros de auditoría, intentos de inicio de sesión y consentimientos de cookies.
- **Almacenamiento de archivos:** Sistema de archivos local dentro del contenedor Docker, ubicado en `/app/documentos/`, para los PDFs subidos por los usuarios.
- **Infraestructura:** Ambiente de ejecución completo definido mediante Docker Compose.
- **Servicio externo:** API de Resend para el envío del código de autenticación de dos factores (2FA).

1.2 Roles de usuario en el sistema

El sistema define los siguientes roles, cada uno con un espacio de trabajo diferenciado:

Rol	Código	Descripción
Paciente	1	Agendamiento de citas y gestión de documentos propios
Cirujano	2	Visualización y gestión de cirugías asignadas
Asistente	3	Apoyo en la gestión de cirugías
Anestesiólogo	4	Visualización de cirugías asignadas

2 Principios de Seguridad

El sistema fue diseñado sobre tres principios fundamentales, alineados con la tríada CIA (*Confidentiality, Integrity, Availability*).

2.1 Confidencialidad

La información solo es accesible para quienes tienen autorización explícita según su rol en el sistema.

La confidencialidad se materializa en la aplicación mediante:

- **Control de acceso basado en roles (RBAC):** Cada endpoint del backend verifica el rol del usuario antes de procesar la solicitud. El frontend redirige automáticamente a cada usuario a su espacio correcto mediante `ProtectedRoute.jsx`.
- **Aislamiento de documentos:** Un paciente solo puede acceder a sus propios documentos. El servicio `documentos.py` verifica que el `paciente_id` del documento coincida con el del usuario autenticado.
- **Hash de contraseñas:** Las contraseñas se almacenan como hashes bcrypt (`auth.py`); jamás en texto plano.
- **Hash del token de sesión:** El JWT no se guarda en la base de datos. En su lugar se almacena su hash SHA-256 en la tabla `sesiones`, de modo que una filtración de la base de datos no compromete los tokens activos.

2.2 Integridad

Los datos solo pueden ser modificados por usuarios autorizados, y toda acción sensible queda registrada en la bitácora de auditoría.

- **Validación en el backend:** Toda entrada se valida en el backend mediante esquemas Pydantic. El frontend nunca es la última línea de defensa.
- **Registro de auditoría:** El servicio `auditoria.py` registra en la tabla `registros_auditoria` cada acción sensible: quién la realizó, sobre qué registro, cuáles fueron los valores anteriores y nuevos, y la dirección IP.
- **Sesión única:** Al crear una nueva sesión, se eliminan todas las sesiones previas del mismo usuario (`sesiones.py`), lo que previene que credenciales robadas sean usadas en paralelo.
- **Tokens de 2FA de un solo uso:** El campo `used` de la tabla `two_factor_challenges` garantiza que cada código solo pueda emplearse una vez.

2.3 Disponibilidad

El sistema debe estar disponible para usuarios legítimos y protegido contra ataques que puedan degradar o interrumpir el servicio.

- **Protección contra fuerza bruta:** El sistema bloquea un correo durante 15 minutos tras 5 intentos de inicio de sesión fallidos consecutivos (`MAX_INTENTOS = 5`, `VENTANA_TIEMPO = 15 min` en `auth.py`).
- **Expiración controlada de sesiones:** Las sesiones expiran según el parámetro `ACCESS_TOKEN_EXPIRE`. Si el servidor detecta expiración del lado de la base de datos, la sesión se elimina automáticamente.
- **Infraestructura dockerizada:** El entorno completo se levanta con un único comando (`docker compose up`), lo que facilita la recuperación ante fallos.

3 Principios de Implementación de Seguridad

3.1 Mínimo privilegio

Cada rol solo posee los permisos estrictamente necesarios para cumplir su función. La aplicación no otorga acceso adicional por omisión. La implementación de `ProtectedRoute.jsx` redirige a los usuarios autenticados hacia su ruta específica según el mapa de roles; cualquier intento de acceder a la ruta de otro rol resulta en una redirección automática a su ruta de rol.

3.2 Seguridad por capas (*Defense in Depth*)

La aplicación implementa controles en múltiples niveles:

1. **Capa de red / infraestructura:** Docker Compose aísla los servicios.
2. **Capa de transporte:** HTTPS en producción (certificado SSL/TLS).
3. **Capa de autenticación:** bcrypt + 2FA por correo electrónico.
4. **Capa de autorización:** Verificación de rol en cada endpoint del backend.
5. **Capa de sesión:** JWT con hash almacenado en BD y expiración dual (criptográfica + servidor).
6. **Capa de datos:** Validación de esquemas Pydantic; consultas con ORM (SQLAlchemy) que previenen inyección SQL.
7. **Capa de auditoría:** Registro centralizado de todas las acciones sensibles con IP y timestamp.

3.3 Fallos seguros (*Fail Secure*)

Ante cualquier error o condición no esperada, el sistema deniega el acceso por defecto:

- Si el JWT es inválido, expirado o su hash no existe en la base de datos, `validar_y_renovar_sesion()` lanza HTTP 401 y no expone ningún detalle técnico interno al cliente.
- Si un challenge de 2FA está marcado como usado, expirado o no existe, se rechaza la solicitud inmediatamente.
- Los mensajes de error devueltos al usuario son genéricos (p.ej., “*Sesión inválida o expirada*”); los detalles técnicos solo se registran internamente.

4 Manejo de Datos Sensibles

4.1 Clasificación de datos

El sistema maneja las siguientes categorías de información sensible:

Tipo de dato	Clasificación	Control aplicado
Contraseñas	Crítico	Hash bcrypt; nunca en texto plano
Datos clínicos (cirugías)	Confidencial	Acceso restringido por rol
Documentos PDF del paciente	Confidencial	Aislados por <code>paciente_id</code> ; almacenados con nombre UUID
Tokens de sesión (JWT)	Crítico	Solo el hash SHA-256 en BD
Tokens 2FA	Crítico	Solo el hash SHA-256 en BD; TTL 10 min
Dirección IP del usuario	Interno	Registrada en auditoría y sesiones
Datos personales (nombre, correo, teléfono)	Confidencial	Acceso según rol
Claves de API (Resend, JWT secret)	Crítico	Variables de entorno (<code>.env</code>)

4.2 Datos de salud

Los datos clínicos (tipo de cirugía, estado, notas médicas, documentos adjuntos) se consideran datos sensibles de salud. Únicamente el personal médico autorizado puede acceder a ellos.

Los pacientes solo acceden a su propia información.

4.3 Documentos PDF

Los archivos subidos por los usuarios se almacenan en el servidor bajo una ruta interna (/app/documentos/{paciente_id}/) con las siguientes garantías (documentos.py):

- **Validación de tipo:** Solo se aceptan archivos con `content_type = "application/pdf"`.
- **Nombre seguro:** El nombre de almacenamiento se genera como `{UUID}_{nombre_original}`, evitando colisiones y ataques de path traversal.
- **Aislamiento por paciente:** Cada paciente tiene su propio directorio.
- **Control de acceso:** Para descargar o listar un documento, se verifica que el `paciente_id` del documento coincida con el del usuario autenticado.
- **Registro de auditoría:** La subida y eliminación de documentos quedan registradas con el ID del usuario, el nombre del archivo y el tipo.

5 Política de Contraseñas

5.1 Almacenamiento

Las contraseñas nunca se almacenan en texto plano. Se utiliza el algoritmo **bcrypt** con factor de costo ajustable (implementado con la biblioteca **bcrypt** de Python). La función `verificar_contraseña()` en `auth.py` normaliza el prefijo del hash (`$2a$` → `$2b$`) para asegurar compatibilidad entre implementaciones.

5.2 Política mínima de contraseñas

- Longitud mínima: **8 caracteres**.
- Debe contener al menos: una letra mayúscula, una letra minúscula, número y un símbolo.
- Las contraseñas se validan en el backend; el frontend puede mostrar indicadores de fortaleza, pero la validación definitiva ocurre en el servidor.

6 Política de Autenticación y Sesiones

6.1 Flujo de autenticación

El sistema implementa un flujo de autenticación en **dos pasos**:

1. **Paso 1 — Verificación de credenciales:**

- El usuario hace escribe correo y contraseña.
- Se verifica bloqueo por intentos fallidos.
- Se compara la contraseña con el hash bcrypt almacenado.
- En caso de éxito, se genera un código 2FA de 6 dígitos (`secrets.randbelow`), se calcula su hash SHA-256 y se almacena en `two_factor_challenges` con TTL de 10 minutos.
- El código se envía al correo del usuario vía Resend.
- Se retorna un `challengeId` (token URL-safe de 32 bytes).

2. Paso 2 — Verificación del código 2FA:

- El usuario envía `challengeId` + código 2FA recibido.
- Se verifica: existencia del challenge, estado `used = false`, no expirado y coincidencia del hash del código.
- El challenge se marca como `used = true`.
- Se crea la sesión en la tabla `sesiones` y se retorna el JWT.

6.2 Gestión de sesiones (JWT + base de datos)

- **Token:** JSON Web Token (JWT) firmado con clave secreta (`settings.SECRET_KEY`) mediante el algoritmo configurado en `settings.ALGORITHM`.
- **Almacenamiento del lado del servidor:** Solo el **hash SHA-256** del JWT se guarda en la tabla `sesiones`. Esto permite la revocación inmediata del lado del servidor.
- **Sesión única:** Al crear una nueva sesión, todas las sesiones previas del mismo usuario se eliminan de la base de datos.
- **Renovación automática:** En cada solicitud autenticada válida, `validar_y_renovar_sesion()` extiende `expira_en` en la base de datos, implementando una ventana de inactividad deslizante.
- **Validación dual de expiración:** Se verifica tanto la expiración criptográfica del JWT como la expiración registrada en la base de datos.
- **Revocación en logout:** `revocar_sesion()` elimina la fila de `sesiones`. El JWT queda huérfano e inutilizable, aunque no haya expirado criptográficamente.

6.3 Almacenamiento del token en el cliente

El token JWT se almacena en `sessionStorage` del navegador (junto con los datos del usuario). El uso de `sessionStorage` garantiza que la sesión se elimine automáticamente al cerrar la pestaña o el navegador.

6.4 Protección contra fuerza bruta

- Máximo **5 intentos fallidos** en una ventana de **15 minutos** por dirección de correo electrónico.
- Los intentos se registran en `intentos_login` con IP y timestamp.
- Al superar el límite se retorna **HTTP 429** con el tiempo de espera.
- Los intentos fallidos se registran también en `registros_auditoria`.

7 Control de Acceso

7.1 Principio de mínimo privilegio

Cada rol recibe únicamente los permisos necesarios para sus funciones. La siguiente tabla resume las operaciones por rol:

Rol	Operaciones permitidas
Paciente	Agendar y ver sus propias citas. Subir, listar y eliminar sus propios documentos PDF. No puede ver datos de otros pacientes.
Cirujano	Ver y gestionar cirugías asignadas. No puede modificar catálogos ni datos de otros usuarios.
Anestesiólogo	Ver cirugías donde está asignado. Acceso de solo lectura a los datos relevantes de cada intervención.
Asistente	Apoyo en gestión de cirugías asignadas. Permisos definidos según las operaciones de su rol en el backend.

7.2 Validación en el backend

Principio crítico: El frontend **nunca** es la última línea de defensa. Toda validación de permisos ocurre en el backend. La redirección por rol en `ProtectedRoute.jsx` es una mejora de experiencia de usuario, no un control de seguridad.

Cada endpoint de la API verifica el token de sesión mediante `validar_y_renovar_sesion()` antes de ejecutar cualquier lógica de negocio. Los servicios como `documentos.py` verifican adicionalmente que los recursos solicitados pertenezcan al usuario autenticado.

8 Protección contra Vulnerabilidades Web

8.1 Inyección SQL

El backend usa **SQLAlchemy ORM** para todas las consultas a la base de datos PostgreSQL. El ORM genera sentencias parametrizadas automáticamente, eliminando la posibilidad de inyección SQL por concatenación de cadenas. Ningún servicio construye consultas SQL dinámicas con entradas del usuario.

8.2 Cross-Site Scripting (XSS)

- React escapa el contenido dinámico por defecto al renderizar componentes, previniendo la inyección de HTML malicioso.
- La aplicación evita el uso de `dangerouslySetInnerHTML`.

8.3 Cross-Site Request Forgery (CSRF)

El sistema utiliza JWT en cabeceras de autorización (**Bearer token**) en lugar de cookies de sesión para las solicitudes autenticadas al API. Este mecanismo previene CSRF de forma inherente, ya que los scripts maliciosos en otros dominios no pueden leer ni adjuntar cabeceras de autorización arbitrarias mediante solicitudes cross-site.

8.4 Cabeceras de seguridad HTTP

Se recomienda configurar las siguientes cabeceras en el servidor:

- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: DENY`
- `Strict-Transport-Security (HSTS)` en producción
- `Content-Security-Policy (CSP)`

9 Registro y Auditoría

9.1 Sistema de auditoría

El servicio centralizado `auditoria.py` registra en la tabla `registros_auditoria` todas las acciones sensibles del sistema. Cada registro incluye:

- `usuario_id`: identificador del actor.

- `accion`: tipo de acción (p. ej., `LOGIN`, `LOGIN_FALLIDO`, `LOGIN_EXITOSO`, `LOGIN_FALLIDO_2FA`, `SUBIR_DOCUMENTO`, `ELIMINAR_DOCUMENTO`, `CREAR_CIRUGIA`).
- `nombre_tabla` y `registro_id`: entidad afectada.
- `valores_anteriores` / `valores_nuevos`: estado antes y después del cambio (formato JSONB).
- `direccion_ip`: IP de la solicitud.
- `creado_en`: timestamp con zona horaria UTC.

9.2 Eventos auditados

- Inicio de sesión exitoso y fallido (incluye fallos en 2FA).
- Subida y eliminación de documentos.
- Creación, modificación y eliminación de cirugías y datos clínicos.
- Cualquier acción sobre catálogos o datos de usuarios.

9.3 Protección de datos en los logs

- Los registros de auditoría **no contienen contraseñas**, tokens JWT ni códigos 2FA.
- Las IPs se registran con fines de seguridad y trazabilidad.
- Los registros de auditoría son de solo escritura desde los servicios; su acceso de lectura está restringido al rol administrador.

10 Configuración Segura del Entorno

10.1 Variables de entorno

Todos los secretos y parámetros sensibles se configuran mediante variables de entorno, no en el código fuente:

- `SECRET_KEY`: clave secreta para firma de JWT.
- `ALGORITHM`: algoritmo JWT (p. ej., `HS256`).
- `ACCESS_TOKEN_EXPIRE_MINUTES`: duración de la sesión.
- `RESEND_API_KEY`: clave de la API de envío de correo.
- `RESEND_API_EMAIL`: correo destino para los códigos 2FA.
- Credenciales de la base de datos PostgreSQL.

10.2 Docker Compose

El entorno de ejecución completo está definido en `docker-compose.yml`, incluyendo la aplicación, la base de datos y cualquier servicio auxiliar. Esto garantiza reproducibilidad y evita que secretos estén hardcodeados en imágenes de contenedores.

10.3 Manejo de errores

- Los mensajes de error devueltos al cliente son genéricos y no revelan detalles de la implementación interna (stack traces, rutas del servidor, versiones de dependencias).
- Los errores detallados se registran únicamente en los logs internos del servidor.

11 Cumplimiento con OWASP Top 10

La siguiente tabla resume el alineamiento del sistema con las categorías del OWASP Top 10:

ID	Categoría OWASP	Control implementado
A01	Control de acceso roto	RBAC en backend; <code>ProtectedRoute</code> en frontend; aislamiento por <code>paciente_id</code> .
A02	Fallos criptográficos	<code>bcrypt</code> para contraseñas; SHA-256 para tokens en BD; JWT firmado.
A03	Inyección	SQLAlchemy ORM con queries parametrizadas; validación Pydantic.
A04	Diseño inseguro	2FA obligatorio; sesión única; principios CIA desde el diseño.
A05	Configuración de seguridad incorrecta	Secretos en variables de entorno; Docker Compose; <code>.gitignore</code> .
A07	Fallos de autenticación	<code>bcrypt</code> ; bloqueo tras 5 intentos; 2FA; revocación de sesión en logout.
A09	Fallas en registro y monitoreo	Auditoría centralizada con IP, usuario, acción y valores antes/después.

12 Referencias

- OWASP. (s.f.). *OWASP Top Ten*. Open Web Application Security Project. <https://owasp.org/www-project-top-ten/>
- OWASP. (s.f.). *Authentication Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP. (s.f.). *Session Management Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- OWASP. (s.f.). *File Upload Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
- Python Cryptographic Authority. (s.f.). *bcrypt documentation*. <https://github.com/pyca/bcrypt/>
- Tiangolo. (s.f.). *FastAPI documentation: Security*. <https://fastapi.tiangolo.com/tutorial/security/>